

C-Bounded Powersort Pseudocodes

Théo Araújo Magalhães

April 10, 2025

TO-DOS:

1. Os comentários para cada algoritmo precisam estar de acordo com as definições usadas no primeiro esboço do artigo (powersort.tex) e precisam ser revisados para fazerem sentido, alguns ainda estão bem abstratos e/ou não descrevem corretamente os algoritmos.
2. Tanto para esse pdf quanto para o anterior, ou seja, para o artigo como um todo, o nome do professor Victor se enquadra como autor? Imagino que sim. Essa pergunta é meio boba.
3. Código para a estrutura de dados que iremos usar.
4. Reestruturar o vai e volta.

Planejamento da estrutura: Guardaremos o início da primeira run, seu fim e seu power com a run anterior (se houver), além disso, cada elemento seguinte será: o fim da run e o power da run com a anterior. run e não teremos um elemento sentinela. O início de uma run baseia-se no fim da anterior + 1. Assim que incluímos um elemento que estoura o tamanho da pilha, temos que esquecer o primeiro elemento e ajustar o novo primeiro elemento para também guardar o seu início, ou seja:

Pilha:

tamanho == 3 run inicio == (inicio1)

1. run1 (fim1, power1)
2. run2 (fim2, power2)
3. run3 (fim3, power3)

lotou!

Inserir run4 (fim4, power4)

esquece run1.

run inicio $\leftarrow$ fim1 + 1

1 $\leftarrow$ (fim4, power4)

(cada pop ou insert decrementa ou aumenta o tamanho)

Para a direita, ACHO QUE NAO PRECISA mudar a estrutura

Introduction: We are very smart and we know things, this is the complaints we have for things that weren't done by us and how we would have made them better.

1 Powersort Versions:

1.1 Standard Powersort

Commentary on Wild and Munro's version: A single pass algorithm that sorts the entire array.

Algorithm 1 Powersort($A[1..n]$) by Wild and Munro:

```
1  $X :=$  stack of runs // capacity  $\lfloor \lg n \rfloor + 1$ 
2  $P :=$  stack of powers // capacity  $\lfloor \lg n \rfloor + 1$ 
3  $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n)$ 
4 while  $e_1 < n$ 
5    $s_2 := e_1 + 1; e_2 := \text{ExtendRunRight}(A[s_2], n)$  //  $A[s_2..e_2]$  next run
6    $p := \text{NodePower}(s_1, e_1, s_2, e_2, n)$  //  $P_j$  for node  $j$  between  $A[s_1..e_1]$ 
   and  $A[s_2..e_2]$ 
7   while  $P.\text{top}() > p$  // previous merge deeper in tree than current
8      $P.\text{pop}()$  //  $\rightarrow$  merge and replace run  $A[s_1..e_1]$  by result
9      $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
10   $X.\text{push}(A[s_1, e_1]); P.\text{push}(p)$ 
11   $s_1 := s_2; e_1 := e_2$ 
12 end while // Now  $A[s_1..s_1]$  is the rightmost run
13 while  $\neg X.\text{empty}()$ 
14    $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
```

1.2 First variation of C-Bounded Powersort (only goes right)

Commentary on our first version: This algorithm performs multiple passes on the array, processing all subtrees of height $C-1$ with each pass. We remind the reader of how Powersort works: It is implicitly a complete binary tree of size $\log n$. Therefore, the max amount of passes needed to sort the array is:

$$\text{N}^\circ \text{ of scans} \leq \frac{\log n}{C-1} \quad (1)$$

Algorithm 2 C-Bounded-Powersort 1($A[1..n]$, C):

```

1  $X :=$  C-bounded stack of runs // capacity  $C$ 
2  $P :=$  C-bounded stack of powers // capacity  $C$ 
3  $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n)$ 
4 while  $e_1 < n$  // This while can be tweaked to use max amount of passes
5   while  $e_1 < n$  // A single pass goes until the end of the array
6      $s_2 := e_1 + 1; e_2 := \text{ExtendRunRight}(A[s_2], n)$ 
7      $p := \text{NodePower}(s_1, e_1, s_2, e_2, n)$  // Using our node power algorithm
8     while  $P.\text{top}() > p$  // We follow the same merge policy
9        $P.\text{pop}()$ 
10     $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
11     $X.\text{push}(A[s_1, e_1]); P.\text{push}(p)$ 
12     $s_1 := s_2; e_1 := e_2$ 
13  end while
14  while  $\neg X.\text{empty}()$ 
15     $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
16    // We're using a limited stack, so we need to check for another pass
17     $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n)$  // If  $e_1 = \text{end}$ , the array is sorted
18 end while

```

1.3 Second variation of C-Bounded Powersort (goes left and right!)

Commentary on our second version: This works rather similarly to our first version. The main difference is the fact that, once we reach the end of the array, instead of restarting our scan from the beginning, we scan, starting from the end, to the left, until we reach the beginning of the array, and, thus, we repeat the scan-right process. This is slightly better than the first version, given that it takes advantage of runs still being in cache instead of restarting from the beginning.

Algorithm 3 C-Bounded-Powersort 1($A[1..n]$, C):

```

1   $X :=$  C-bounded stack of runs
2   $P :=$  C-bounded stack of powers
3   $Side := L$  Direction of the scan           // Starts from left to right
4   $s_1 := 1; e_1 = ExtendRunRight(A[1], n)$ 
5  while  $e_1 < n$ 
6    if  $Side == L$ 
7      while  $e_1 < n$ 
8         $s_2 := e_1 + 1; e_2 := ExtendRunRight(A[s_2], n)$ 
9         $p := NodePower(s_1, e_1, s_2, e_2, n)$ 
10       while  $P.top() > p$ 
11          $P.pop()$ 
12          $(s_1, e_1) := Merge(X.pop(), A[s_1..e_1])$ 
13        $X.push(A[s_1, e_1]); P.push(p)$ 
14        $s_1 := s_2; e_1 := e_2$ 
15     end while
16     while  $\neg X.empty()$ 
17        $(s_1, e_1) := Merge(X.pop(), A[s_1..e_1])$ 
18      $e_1 = ExtendRunLeft(A[n], 1)$  // Now, we're scanning from right to left!
19     if  $e_1 == 1$ 
20       break // If we could extend until the beginning, our run is the entire array.
21      $Side \leftarrow R$  // Side of the next scan
22   else if  $Side == R$ 
23     while  $e_1 > 1$  // We're scanning from n down to 1
24        $s_2 := e_1 - 1; e_2 := ExtendRunLeft(A[s_2], 1)$ 
25        $p := NodePower(s_1, e_1, s_2, e_2, n)$ 
26       while  $P.top() > p$ 
27          $P.pop()$ 
28          $(s_1, e_1) := Merge(X.pop(), A[s_1..e_1])$ 
29        $X.push(A[s_1, e_1]); P.push(p)$ 
30        $s_1 := s_2; e_1 := e_2$ 
31     end while
32     while  $\neg X.empty()$ 
33        $(s_1, e_1) := Merge(X.pop(), A[s_1..e_1])$ 

```

```
34       $s_1 := 1; e_1 = \textit{ExtendRunRight}(A[1], n)$  // Now, we're scanning from  
    left to right!  
35      if  $e_1 == n$   
36          break // If we could extend until the end, our run is the entire array.  
37       $Side \leftarrow L$  // Side of the next scan
```

TEMPORARY VERSION OF OUR SECOND VERSION TO APPLY CHANGES:

1.4 Second variation of C-Bounded Powersort (goes left and right!)

Commentary on our second version: This works rather similarly to our first version. The main difference is the fact that, once we reach the end of the array, instead of restarting our scan from the beginning, we scan, starting from the end, to the left, until we reach the beginning of the array, and, thus, we repeat the scan-right process. This is slightly better than the first version, given that it takes advantage of runs still being in cache instead of restarting from the beginning.

Algorithm 3 C-Bounded-Powersort 1($A[1..n]$, C):

```

1   $X :=$  C-bounded stack of runs
2   $P :=$  C-bounded stack of powers
3  while true
4     $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n)$ 
5    while  $e_1 < n$ 
6       $s_2 := e_1 + 1; e_2 := \text{ExtendRunRight}(A[s_2], n)$ 
7       $p := \text{NodePower}(s_1, e_1, s_2, e_2, n)$ 
8      while  $P.\text{top}() > p$ 
9         $P.\text{pop}()$ 
10      $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
11      $X.\text{push}(A[s_1, e_1]); P.\text{push}(p)$ 
12      $s_1 := s_2; e_1 := e_2$ 
13   end while
14   while  $X.\text{size} > 1$ 
15      $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
16    $e_1 = \text{ExtendRunLeft}(A[n], 1)$  // Now, we're scanning from right to left!
17   if  $e_1 == 1$ 
18     break // If we could extend until the beginning, our run is the entire array.
19   while  $e_1 > 1$  // We're scanning from n down to 1
20      $s_2 := e_1 - 1; e_2 := \text{ExtendRunLeft}(A[s_2], 1)$ 
21      $p := \text{NodePower}(s_1, e_1, s_2, e_2, n)$ 
22     while  $P.\text{top}() > p$ 
23        $P.\text{pop}()$ 
24      $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 
25      $X.\text{push}(A[s_1, e_1]); P.\text{push}(p)$ 
26      $s_1 := s_2; e_1 := e_2$ 
27   end while
28   while  $X.\text{size} > 1$ 
29      $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1])$ 

```

```

30    $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n)$  // Now, we're scanning from left
    to right!
31   if  $e_1 == n$ 
32       break // If we could extend until the end, our run is the entire array.

```

2 NodePower versions:

2.1 Standard Nodepower:

Commentary on Wild and Munro's Node Power: This is so silly and expensive!

Algorithm 4 NodePower(s_1, e_1, s_2, e_2, n):

```

1   $n_1 := e_1 - s_1 + 1; n_2 := e_2 - s_2 + 1; \ell := 0$ 
2   $a := (s_1 + n_1/2 - 1)/n; b := (s_2 + n_2/2 - 1)/n$ 
3  while  $\lfloor a \cdot 2^\ell \rfloor == \lfloor b \cdot 2^\ell \rfloor$  do  $\ell := \ell + 1$  end while
4  return ( $\ell$ )

```

2.2 Our Nodepower:

Commentary on our Node Power: Ours is cool and epic. There is a catch, though. Our Node Power doesn't clearly define a Merge Tree, sometimes one run will share the same power between its two neighbors. This can be ignored and simply solved by changing our merge policy to:

$$\mathbf{While} P.top() \geq p \quad (2)$$

instead of:

$$\mathbf{While} P.top() > p \quad (3)$$

Algorithm 4 NodePower(s_1, e_1, s_2, e_2, n):

```

1   $\text{xor} := s_1 \wedge s_2$ 
2   $\text{first\_bit} := \_builtin\_clz(\text{xor})$ 
3   $\ell := \text{first\_b}$ 
4  return ( $\ell$ )

```
